



OPEN Securing IoT networks: a machine learning approach for detecting unusual traffic patterns

Nadeem Sarwar¹✉, Raed S. Alharthi²✉, Mansourah Aljohani³ & Mostafa A. Elhosseini^{4,5}

The exponential growth of the Internet of Things (IoT) poses substantial security challenges due to its heterogeneous, decentralized nature. This paper introduces a machine learning (ML) framework to enhance IoT network security by identifying and mitigating anomalous traffic patterns. The combination of the NBaIoT and UNSW-NB15 datasets enabled us to construct a complete environment for analysis. We used an extensive data preprocessing pipeline, including cleaning, integration, transformation, and feature engineering, to prepare the datasets for ML. We constructed a framework that was assessed with several types of ML models and achieved good results: the decision tree model with support of 95% accuracy and 93% precision with 92% recall and F1 score of 92.5; the SVM model with the backing of 96% accuracy and 94% precision with 92 and 95 recall and F1 score respectively; the random forest model with support of 97% accuracy and 96% precision with 92 and 95 recall and F1 score respectively. The best performance of our neural network model was 98% precision, 97% recall, and 97.5% perception, respectively. These findings indicate that our neural network-based framework is more effective at detecting and preventing IoT traffic anomalies than other models, in both accuracy and overall efficiency. The results highlight the potential of using ML to deliver robust, scalable, real-time anomaly-detection solutions for an IoT network. Further studies will aim to improve it by implementing the model in practice and incorporating additional datasets and machine-learning methods to make the model more flexible and resilient.

Keywords Anomaly detection, Decision trees, Data preprocessing, Feature engineering, IoT security, Machine learning, Network security

Because of the rapid development of the Internet of Things (IoT), contemporary society has changed, as billions of interconnected devices can now exchange vast amounts of data without issue. This is a massive integration that is bound to be more efficient and convenient across a wide range of areas, including smart homes and industrial automation¹. Nonetheless, the security risks have also been enhanced by the dramatic growth of the IoT ecosystems. IoT infrastructure is becoming more susceptible to attacks, as any compromised device can interfere with the network or serve as a gateway for larger cyberattacks. IoT environments are distributed, heterogeneous, and dynamic, and are often not well-protected by traditional security mechanisms². It is also related to the fact that the devices are not similar, and each has security vulnerabilities. One can see that these problems can be managed through the use of Machine learning (ML), which offers an adaptive approach to analyzing and learning from the information to conclude deviant behavior, an indicator of security threats^{3,4}. This study aims to implement an ML approach to improve the security of Internet of Things networks against such threats, whilst maintaining a highly dynamic defense system. Several challenges are associated with the use of ML to protect IoT networks. Given the disparities in device capabilities, the optimal anomaly-detection model should be highly versatile. Besides, the mobility of IoT networks and the inbound and outbound connectors necessitate that security systems be continuously updated to accommodate new devices. Several IoT devices are capable of executing only a small number of ML models in real time due to their limited computational capabilities⁵. Moreover, the privacy concerns posed by the various data generated by low-cost IoT appliances

¹Department of Computer Science, Bahria University Lahore Campus, Lahore 54600, Pakistan. ²Department of Computer Science and Engineering, University of Hafr Al-Batin, Hafar Al-Batin 39524, Saudi Arabia. ³Department of Computer Science, College of Computer Science and Engineering, Taibah University, Yanbu 46421, Saudi Arabia. ⁴Department of Information Systems, College of Computer Science and Engineering, Taibah University, Yanbu 46421, Saudi Arabia. ⁵Computers and Control Systems Engineering Department, Faculty of Engineering, Mansoura University, Mansoura 35516, Egypt. ✉email: nadeem_srwr@yahoo.com; ralharthi@uhb.edu.sa

should be balanced against security needs. The study will provide an ML-based method to identify and counteract suspicious traffic on IoT networks, while ensuring the best time and preserving privacy.

The main goal of this study is to develop a machine learning system to identify and prevent abnormal traffic patterns in IoT systems. The contributions of this work are the following:

- To review and synthesize existing literature on IoT security and anomaly detection to establish a comprehensive background for the study.
- To identify the most effective machine learning algorithms for anomaly detection in IoT traffic, considering IoT devices and networks' unique characteristics and constraints.
- To design and implement a prototype system that uses selected machine learning algorithms to detect anomalous traffic in a controlled IoT environment.
- To evaluate the performance of the prototype system through rigorous testing with real-world IoT traffic data.
- To propose recommendations for deploying machine learning-based anomaly detection systems in various IoT scenarios, including smart homes, healthcare, and industrial settings.

The study aims to detect anomalies in IoT network traffic using machine learning algorithms. It is oriented towards general IoT protocols and devices, offering practical, scalable solutions with real-world applications. The study will exclude the physical security of the IoT devices and the development of novel machine-learning algorithms. Nevertheless, it will consider how the available algorithms can be applied in the IoT environment. The given paper presents an IoT anomaly-detection framework based on machine learning, optimized for real-world deployment by incorporating the NBaIoT and UNSW-NB15 datasets to improve generalization. To enhance detection accuracy and reduce computational resources, we consider and adopt advanced feature selection algorithms (PCA, Information Gain, Correlation Analysis), which make the model suitable for resource-constrained IoT environments. Our research is a comprehensive empirical benchmarking analysis of Decision Trees, SVMs, Random Forests, and Neural Networks, examining trade-offs among accuracy, efficiency, and scalability. Our proposal offers specifics of IoT deployment, namely edge computing viability, adaptive learning, and federated security models, and guarantees real-time threat detection at a resource overhead lower than usual. The study has limitations, including data that reflect current trends in IoT traffic and the likelihood of rapid changes in IoT technology; thus, the findings may no longer be relevant. Also, the study will be limited by the computing power available to process data and train machine learning models. Section “Related work” reviews related work, summarizing existing research on IoT security and machine learning applications to identify research gaps. The methodology section describes the research design, namely market data selection, data preprocessing, feature engineering, and machine learning model selection. The section on the experiment and results describes the combination of real and synthetic IoT traffic data, the results of model training and validation, and comments on the performance of different models relative to one another. The paper concludes with a discussion of its significant findings, contributions to IoT security, and recommendations for future research.

Related work

This section discusses the literature that forms the basis of this study and the security threats to IoT and networks, including anomalous traffic. There are gaps in the state of research in this expansive area, which examines the strengths and weaknesses of existing strategies and outlines the areas where machine learning has been utilized to achieve the following purposes. The security of IoT encompasses a broad range of data protection challenges, including network stability and the reliability of IoT devices. The scale and connectivity of the IoT networks, combined with the inherent security of low-cost, resource-limited devices, frequently exacerbate the challenges^{6,7}. The fact that IoT is widespread across many industries only creates more difficulties in developing a network security protocol^{8,9}. Connected devices are not the only objects concerned with IoT security; the data they gather and share are as well. It is also important to note that, with the general increase in the adoption of IoT solutions and their implementation in essential spheres, including healthcare and the protection of critical infrastructure, the number of security threats and their consequences seems enormous¹⁰. According to the literature, the feeling of a secure system that can efficiently function in the decentralized, frequently ad hoc structure of IoT networks, which are not efficiently handled by traditional security, is a dire requirement as well¹¹. The identification of anomalies in IoT networks is a research issue gaining popularity, as anomalies can indicate a variety of threats, including technical flaws, cyber espionage, and sabotage. The incorporation of IoT networks, in which devices often communicate independently of human intervention, makes the identification of such anomalies quite challenging^{12,13}. The literature shows that, whereas IoT networks collect vast amounts of data, how best to handle this data and identify patterns that raise alarms remains unclear^{14,15}. Research has proposed applying statistical models, machine learning, and data mining methods for detecting anomalies in network traffic, stressing the requirement of the proposed models for high adaptability due to the constantly increasing use of IoT devices^{16,17}. Because IoT environment-based systems operate in dynamic environments, they must identify known anomalies and predict future, unknown, or emerging traffic pattern-based anomalies^{18,19}. This results in isolation; the high dimensionality of data is a well-known problem in the anomaly detection system, as the high dimensionality causes a more significant number of features and, therefore, decreases the efficiency of the system; thus, feature selection and dimensionality reduction are usually used to help enhance the speed and accuracy of the anomaly detection systems²⁰. Furthermore, the ratio between these errors of the first and second kinds is an essential factor in designing these systems, as both degrade the IoT network's usability and reliability²¹. In the context of network security, specifically in the IoT, the field of machine learning (ML) use is a new and dynamic field that seeks to address the many issues of Anomaly detection and Security Systems. AI algorithms have been widely adopted to enhance security systems, detect intrusions, and manage diverse threats

in real time. ML algorithms are ideal for pattern and anomaly detection because these are common scenarios in large, diverse traffic on IoT networks. One of the enormous benefits of using ML algorithms is the ability to regularly update security measures to respond to emerging tactics by evildoers^{9,22}. The supervised category includes neural networks and support vector machines, which have been used earlier to distinguish between normal and malicious traffic, as classification requires labeled datasets²³. In return, unsupervised learning is most appropriate when labeled data are hard to obtain or unavailable. Some traditional approaches, such as clustering or anomaly detection, can identify potential security threats without concern for the type of traffic expected from an attacker²⁴. This is very important for IoT scenarios, where new kinds of connected devices and, therefore, new traffic flows may appear at any time. Another narrowly defined subset of ML – deep learning, associated with neural networks with a large number of layers – holds potential for detecting subtle details and makes it difficult for other ML methods to identify cyber threats²⁵. More specifically, we will see that deep learning models can process raw inputs without extensive feature engineering, thereby lowering operational costs and the amount of specialized knowledge required to set up security systems successfully. However, the use of ML in network security is not without some problems, as discussed below. Training complex models may require substantial computational power, which is often unattainable on IoT devices due to cost constraints. There is also a data privacy problem, since the ML model is trained on private and sensitive data²⁶. Nevertheless, the paper has found that the literature provides more evidence of a progression toward conceptualizing ML as a means of transforming IoT security. ML-based systems can effectively protect the rapidly growing IoT environment through automated heap identification and form the basis for subsequent security actions.

There have been changes to approaches to IoT security over the years as various attempts have been made to address the complex issues associated with integrating the physical and cyber realms. In the past, security was mainly an add-on to the IoT that borrowed security concepts from the information technology security domain, which was primarily oriented toward protecting endpoints from viruses, encrypting information, and authenticating users. Still, certain features of IoT ecosystems, including size, diversity, and limited resources, became so pronounced that such traditional approaches were no longer sufficient²⁷. Transitioning to the evolution of strategy plans as the IoT environment expanded, new security protocols were adopted to align with the lightweight nature of IoT devices on the market. One is Datagram Transport Layer Security (DTLS), designed for devices with limited computational capabilities and power supply, such as IoT devices. It means it provides end-to-end connectivity and encryption suitable for IoT settings^{17,28}. IoT-specific intrusion detection systems (IDS) have also been developed to detect anomalies and familiar attacks on IoT networks. Such systems track all network transactions and provide corresponding alarms if there is any possible malicious activity. They can be used to address dominant IoT threats, such as DDoS and MitM attacks, which stem from the open, distributed nature of most IoT devices²⁹. According to new research, the trend is toward increasingly comprehensive security systems that incorporate both product and program components. This practice will involve hosting secure microcontrollers and TPMs to provide a physical root of trust, and adopting sophisticated software to assess threats and act accordingly in a continuous manner. The multi-layered defense system to be applied should be based on integrating solutions capable of addressing emerging threats in the IoT³⁰. Nevertheless, artificial intelligence and machine learning, which are part of IoT security, have been discovered recently. Using one of the primary qualities of machine learning, which is the possibility to predict new events in the future, the security systems can identify threats even before they happen and modify security levels according to the current trends in past incidents³¹. Recent research illustrates the importance of increasing the significance of high-resolution and context-aware architectures and generative feature-enhancement models. Indicatively, the context-sensitive high-resolution network suggested in³² enhances semi-supervised segmentation by leveraging multi-scale contextual representations. This method is gaining increasing popularity in IoT traffic modeling, where contextual dependencies are essential. On the same note, Fast-DSAGCN³³ proposes multi-facet attention mechanisms that can improve representation learning and apply to complex IoT anomaly patterns that require accurate feature extraction. GAN-augmentation is also taking center stage. In addition to segmentation, recent works in the field of IoT pay attention to deep optimization techniques, e.g., reinforcement-based intrusion detection³⁴ and hybrid CNN-GNN systems that incorporate temporal-spatial correlations^{35,36}. The consideration of these developments reinforces the argument for the novelty of our framework, as our combination of multistage feature selection and ML-based anomaly detection is not only in line with but also mirrors the current trends in deep learning studies in the field of IoT security.

Nevertheless, in line with these developments, the literature indicates that there remains a strong demand to enhance the security of IoT further. The unlimited and evolving characteristics of IoT networks, along with the growing sophistication of hackers in attacking data, require a preventive, complex security strategy. Future research focuses on developing self-healing solutions, implementing blockchain for security, and examining novel cryptographic solutions compatible with IoT, given its large, complex environment.

Methodology

The planning of research to control IoT networks against abnormal traffic flow using ML methodology comprises the following features. The approach explicitly engages the research objectives, suppressing every process to scientifically and empirically sound stages. The flow of this research is indicated in Fig. 1. This is usually about selecting the correct machine learning models, acquiring and preparing data, training and validating the models, and assessing their ability to identify abnormal cases.

Dataset

This section discusses the combination of the N_BaIoT and UNSW-NB15 datasets, creating a vast, coupled dataset for developing and training specialized machine learning algorithms for IoT security. The datasets

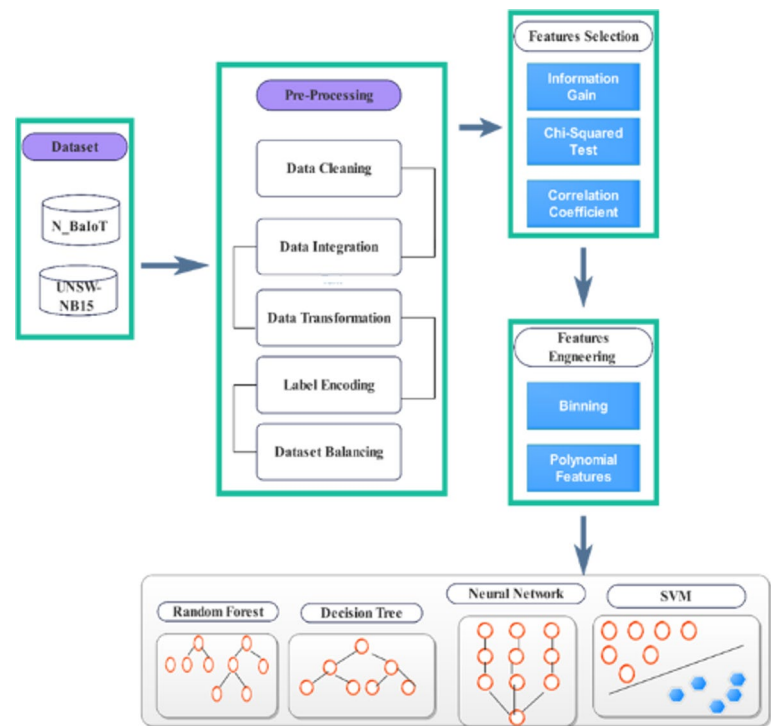


Fig. 1. Proposed methodology framework.

Dataset	Source	Description	Attacks covered
N_BaIoT	Ben-Gurion University	Network traffic from various IoT devices under multiple attack scenarios	Botnet, DDoS, DoS
UNSW-NB15	University of New South Wales	A dataset with labeled network logs, including normal and attack behaviors	Exploits, Fuzzers, Reconnaissance, and others

Table 1. Description of N_BaIoT and UNSW-NB15 datasets.

embodied in Table 1 are introduced as key components of the research approach, serving as the basis for creating and testing the ML methods appropriate for IoT security.

The N_BaIoT dataset, therefore, contains information on the network traffic of numerous IoT devices under attack. The dataset is segmented into regular and intrusive traffic, which is important for system accuracy in supervised learning and for anomaly detection. The UNSW-NB15 dataset is a large, realistic network log dataset and was previously divided into standard and attack behavior categories across various attack types. The types of attacks in this data are numerous, posing a significant challenge for the anomaly detection algorithm. These datasets form the empirical basis of the research and enable the study of how effectively machine learning can identify and mitigate the threats posed by IoT networks. The fact that the N_BaIoT dataset is specialized for attack-specific IoT enhances the study’s applicability to the domain. Conversely, the UNSW-NB15 dataset provides a more general outlook of threats to network security. These datasets are crucial for fine-tuning the algorithms and the proof-of-concept models developed; hence, they depict real-life IoT security challenges. Thus, the N_BaIoT and UNSW-NB15 datasets were used in this work because they balance IoT-specific and overall network traffic features. To our knowledge, there are no other similar IoT traffic datasets with corresponding attack samples like DDoS or botnets as N_BaIoT offers, making it very suitable for evaluating the IoT security models. UNSW-NB15 expands coverage by recognizing a variety of cyber-attacks, including exploits and reconnaissance, enabling it to detect anomalies across different network types. However, there is no non-IoT traffic in N_BaIoT, and thus several traffic features present in UNSW-NB15 may not accurately reflect the actual characteristics of IoT networks. The proposed model combines all these datasets to ensure its full coverage and independence to support its ability to identify any in any network setting. To create a unified dataset suitable for training and evaluation, we merged the N_BaIoT and UNSW-NB15 datasets by aligning their feature schemas. Common attributes such as IP addresses, ports, protocol types, and traffic features were retained, while non-overlapping or redundant fields were excluded or mapped appropriately. To bring both datasets to the same schema, we retained only standard, semantically valid features, e.g., flow statistics and protocol types. To merge the N_BaIoT and UNSW-NB15 datasets, we first standardized their shared traffic characteristics and removed fields of domain

Input:Raw datasets $D_1 \leftarrow \text{N_BaIoT}, D_2 \leftarrow \text{UNSW-NB15}$ **Output:**Trained a machine learning model M^* for anomaly detection

```

1:  $D \leftarrow \text{IntegrateDatasets}(D_1, D_2)$     $\boxtimes$  merge, align schema, remove domain-specific fields (IP, timestamp)
2:  $D \leftarrow \text{CleanData}(D)$     $\boxtimes$  impute missing values (mean/median), remove IQR-based outliers
3: // Feature scoring and top-k selection
4: for each feature  $f_i \in D$  do
5:    $IG_i \leftarrow \text{InformationGain}(f_i, y)$ 
6:    $\chi_i^2 \leftarrow \text{ChiSquare}(f_i, y)$ 
7:    $\rho_i \leftarrow |\text{PearsonCorrelation}(f_i, y)|$ 
8:    $s_i \leftarrow \text{norm}(IG_i) + \text{norm}(\chi_i^2) + \text{norm}(\rho_i)$     $\boxtimes$  combined normalized score
9: end for
10: Sort features in descending order of  $s_i: \{f_{(1)}, \dots, f_{(m)}\}$ 
11: Find smallest  $k$  such that  $\frac{\sum_{j=1}^k s_{(j)}}{\sum_{j=1}^m s_{(j)}} \geq 0.90$ 
12:  $F_{\text{sel}} \leftarrow \{f_{(1)}, \dots, f_{(k)}\}$     $\boxtimes$  selected top-k features
13: // Data transformation and split
14: Apply Z-score normalization to all numeric features in  $F_{\text{sel}}$ 
15: Encode categorical features using One-Hot or Label Encoding
16:  $(D_{\text{train}}, D_{\text{test}}) \leftarrow \text{TrainTestSplit}(D, \text{ratio} = 0.8)$ 
17: // Class balancing (training only)
18: if  $\text{IsImbalanced}(D_{\text{train}})$  then
19:    $D_{\text{train}} \leftarrow \text{SMOTE}(D_{\text{train}})$ 
20: end if
21: // Dimensionality reduction with PCA
22:  $F_{\text{PCA}} \leftarrow \text{PCA}(F_{\text{sel}}, \text{varThreshold} = 0.95)$     $\boxtimes$  retain PCs explaining  $\geq 95\%$  variance
23: // Model training and validation
24: for each model  $M \in \{\text{DT}, \text{SVM}, \text{RF}, \text{NN}\}$  do
25:    $M \leftarrow \text{GridSearchCV}(M, D_{\text{train}}[F_{\text{PCA}}], 5\text{-fold})$ 
26:    $\text{Eval}_M \leftarrow \text{Evaluate}(M, D_{\text{test}}[F_{\text{PCA}}])$     $\boxtimes$  Accuracy, Precision, Recall, F1
27: end for
28:  $M^* \leftarrow \arg \max_M \text{F1}(\text{Eval}_M)$  with consideration of inference time/efficiency
29: return  $M^*$ 

```

Algorithm 1: IoT Traffic anomaly detection—preprocessing and model training.

specificity, including IP addresses and device identifiers. Any common features were standardized to a uniform form, and the class labels were standardized into a binary variable (normal vs. anomalous) to produce a single dataset that could be jointly preprocessed and trained.

Data preprocessing

The first and most likely the most crucial step of our preprocessing pipeline was data cleaning, which dealt with the missing values, the outliers, and other discrepancies in the data. To ensure that the following results were unbiased, we applied several imputation methods, distribution-based outlier and influential observation identification methods, and internal consistency checks to the datasets. All preprocessing steps, such as normalization and encoding, was performed equally to align features and ensure domain compatibility. Correlation analysis and information gain were used to select features with high predictive value and low redundancy, thereby mitigating cross-domain variance. Data cleaning involves removing irrelevant or redundant features and correcting inconsistencies. Given a dataset D , irrelevant features F_{ir} are removed to form a cleaned dataset D' :

$$D' = D / F_{ir} \quad (1)$$

where F_{ir} represents features such as timestamps or IDs that do not contribute to anomaly detection. Error correction is applied through automated rules or validation functions ϕ , ensuring each feature f_i adheres to valid ranges:

$$f'_i = \phi(f_i)$$
(2)

where ϕ is the correction function. Let D be the dataset with missing values $f_{ij} = NaN$. Missing values are either imputed or removed: Imputation (mean or median):

$$f'_{ij} = \frac{1}{n} \sum_{k=1}^n f_{ik}, \text{ or } f'_{ij} = median(f_i)$$
(3)

This fills the missing value with the feature’s mean or median.
Deletion:

$$D' = D / \{f_v : f_{ij} = NaN \text{ for many } j\}$$
(4)

This removes rows in which a significant portion of the values are missing.
Outliers are identified using the standard deviation or interquartile range (IQR). For a feature f_i , an outlier is defined as.

$$f_{ij} \text{ is an outlier if } |f_{ij} - \mu| > k\sigma$$
(5)

where μ is the mean, σ is the standard deviation, and k is a threshold (typically $k = 3$).

$$f_{ij} \text{ is an outlier if } f_{ij} \notin [Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$$
(6)

where $Q1$ and $Q3$ are the first and third quartiles, and IQR is the interquartile range. Outliers can be removed or capped to fall within acceptable limits:

$$f'_{ij} = max(min(f_{ij}, Q3 + 1.5 \times IQR), Q1 - 1.5 \times IQR)$$
(7)

Algorithm 1 outlines the complete pipeline for IoT anomaly detection using machine learning. It begins by integrating two benchmark datasets (N_BaIoT and UNSW-NB15), then proceeds to data cleaning, feature selection, normalization, and balancing to prepare the data for model training.
This section outlines the preprocessing tasks performed to ensure data quality of the main dataset and readiness for the machine learning models presented in Table 2.
The initial preprocessing phase involves cleaning the data by removing irrelevant features, such as timestamps that do not contribute to anomaly detection, and correcting any errors in the data entries. Missing values are handled either by removing the affected records or by estimating their values using statistical imputation.

$$D' + D \text{ }[:, \text{Frelevant}]$$
(8)

$$D''_{ij} = \phi(D_{ij})$$
(9)

$$\frac{\sum_{i=1}'' (D_{ij} \neq missing) \cdot D_{ij}}{\sum_{i=1}^n (D_{ij} \neq missing)}$$
(10)

Step	Description	Techniques used	Tools/Software
Data Cleaning	Removing irrelevant data, correcting errors, and handling missing values	Filtering, Imputation	Python (Pandas, NumPy)
Data Integration	Combining data from multiple sources to create a consistent dataset	Merging, Joining	Python (Pandas)
Data Transformation	Normalizing and scaling data to a uniform format	Min-Max Scaling, Z-score Normalization	Python (Scikit-learn)
Data Reduction	Reducing data volume but producing the same or similar analytical results	Dimensionality Reduction (PCA), Feature Selection	Python (Scikit-learn)
Data Discretization	Converting continuous features into discrete bins	Binning, Histogram Analysis	Python (Pandas, Matplotlib)
Feature Engineering	Creating new features to improve model performance	Feature Extraction, Feature Construction	Python (Feature-engine)
Label Encoding	Transforming categorical labels into a numerical format	Label Encoding, One-Hot Encoding	Python (Scikit-learn)
Dataset Balancing	Addressing the imbalance in the dataset to prevent model bias	Oversampling (SMOTE), Under-sampling	Python (Imbalanced-learn)

Table 2. Data preprocessing steps.

Input:

Numerical feature set $F = \{f_1, f_2, \dots, f_n\}$ from datasets D_1 and D_2

Output:

Scaled feature set $F' = \{f'_1, f'_2, \dots, f'_n\}$ with standardized values

1. **Initialize:** Let $F' \leftarrow \emptyset$

2. **For each feature $f_i \in F$:**

If f_i is numeric:

Compute mean μ_i and standard deviation σ_i of f_i

Normalize using Z-score:

$$f'_i \leftarrow \frac{f_i - \mu_i}{\sigma_i}$$

Append f'_i to F'

3 **Return scaled feature set F'**

Algorithm 2: Feature scaling and normalization for IoT traffic data.

$$D'''_{ij} = \begin{cases} D_{ij} & \text{if } D_{ij} \neq \text{missing} \\ \mu_j & \text{if } D_{ij} = \text{missing} \end{cases} \quad (11)$$

$$D_{reduced} = \{r_i \in D \mid \forall j, D_{ij} \neq \text{missing}\} \quad (12)$$

where D is the original dataset, D' is the dataset after the removal of the relevant feature, D'' is the dataset after error correction, D''' is the dataset after missing value imputation, $D_{reduced}$ is the data set after deleting a record with a missing value, $F_{relevant}$ is the set of indices for a relevant feature, and ϕ is the error correction function.

Given that the N_BaIoT and UNSW-NB15 datasets originate from different sources, it is essential to integrate them into a unified format. This process ensures the combined dataset is consistent and can effectively train and test the machine-learning models.

The mathematical representation of data integration, where two datasets, A and B, are combined into a unified dataset, C, can be expressed as a union of the two sets with a function to resolve any inconsistencies:

$$C = \text{integrate}(A, B) \quad (13)$$

This function can be detailed as follows:

Let A be a set of m features from the N_BaIoT dataset, and B be a set of n features from the UNSW-NB15 dataset.

Define a mapping function F that aligns the features of A and B based on a common schema S.

The integrated dataset C is then the concatenation of A and B over the common schema S:

$$C = \{(a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_n) \mid (a_1, a_2, \dots, a_m) \in A, (b_1, b_2, \dots, b_n) \in B, f(A, B) \rightarrow S\} \quad (14)$$

In practice, this involves creating a new dataset where each record consists of data from A and B, with a function f applied to ensure consistency in the data types and scales.

The datasets are transformed to ensure that all the numerical features have a similar scale. This is important for algorithms sensitive to the magnitude of the data, such as distance-based and gradient-descent algorithms.

Algorithm 2 normalizes all numerical features in datasets A and B by calculating their mean and standard deviation, then scaling each feature to produce the scaled datasets A' and B' .

To improve computational efficiency, techniques like Principal Component Analysis (PCA) and feature selection methods reduce the number of features in the dataset while retaining the most informative ones.

$$\sum = \frac{1}{n-1} X^T X \quad (15)$$

$$Y = X P \quad (16)$$

Where $\sum$ is the covariance matrix, X is the standardized data matrix, P is the matrix of selected.

Continuous features are discretized into bins to simplify analysis and identify patterns that are not apparent in the raw data. New features are engineered from the existing data to enhance the predictive power of the machine learning models. This may include aggregating feature creating interaction terms, or decomposing complex data structures into simpler components. Categorical features are encoded as numerical values that machine learning algorithms can interpret. Techniques such as label encoding and one-hot encoding are used depending on the nature of the categorical data.

The distribution of classes in the two sets is discussed. If an imbalance is observed, the measures employed include the Synthetic Minority Over-sampling Technique (SMOTE) or under-sampling.

All these preprocessing steps are meant to prepare the datasets for the Machine learning tasks that are to follow. They ensure that models are fed with the correct data and that this data is as clean as possible regarding structure.

Feature selection and engineering

Feature selection and engineering are essential and relevant steps in the data preprocessing phase of machine learning models. This section describes choosing the most informative features and converting them into formats that dramatically enhance model performance. Feature selection was performed on the primary dataset to retain only the most essential features for accurate model prediction. The method employed is Information Gain, which measures a feature’s worth by quantifying the information gained about the target variable. The Chi-Square Test is a statistical tool to test the hypotheses and examine the relationship between two variables. Features whose values depend heavily on the response are used to train the models. Correlation Coefficient is used to select features highly correlated with the target variable, while comparatively low inter-correlation among features is preferred to avoid dimensionality.

Feature engineering methods

Feature engineering involves creating and modifying new features to enhance model interpretability and performance. The methods applied include binning and polynomial features. In binning, continuous features were categorized into bins, which simplifies the model and can reveal non-linear patterns. In polynomial features, interaction terms were created to capture the relationship between different features.

Dimensionality reduction

Dimensionality reduction was applied to reduce the feature space and improve computation time. Principal Component Analysis (PCA) was used to transform the features into a set of linearly uncorrelated variables called principal components.

Data representation

The main dataset representation was optimized for the machine learning algorithms. For instance, categorical variables were encoded using one-hot encoding, and normalization was applied to ensure all features contribute equally to the distance calculations in algorithms. In this work, PCA, Information Gain, and Correlation Analysis were applied systematically to identify the most relevant features for anomaly detection. Each feature was first scored independently using Information Gain and Chi-Square to measure its statistical contribution to the target label. At the same time, Pearson correlation quantified its linear relationship with anomalous behavior. Table 3 summarizes the features selected using different selection techniques and their respective scores for the main dataset.

Figure 2 illustrates the percentage of variance explained by each principal component derived from PCA for the main dataset.

For the primary dataset, a heatmap visualizing the importance scores of features across different selection methods is presented in Fig. 3.

Machine learning model selection

Selecting a machine learning algorithm to secure IoT networks against anomalous traffic involves evaluating algorithms against performance criteria. The selection was based on the following criteria. The algorithm’s ability to correctly classify traffic as normal or anomalous. The time and resources required by the algorithm to train and make predictions. This is typical for IoT applications: the more significant the amount of data generated daily, the greater the difference in the algorithm’s performance. The algorithm’s strength is adapting to the computation and still delivering optimal results with noisy or missing data. All these Decision Tree, SVM, Random Forest, and Neural Network models have proven vital for IoT traffic anomaly detection for several reasons, as outlined below. Decision trees are easily interpretable, and their training is relatively fast, making them suitable for rapid, conventional, rule-based pattern recognition. SVM outperforms other models in handling non-linear patterns in higher dimensions, which are critical in IoT data. Random Forest is a way to increase flexibility: the algorithm uses several trees instead of one, applying an averaging to reduce overfitting and improve accuracy when dealing with noisy data. Neural Networks are selected for their high potential to identify intricate, non-linear correlations, which makes them effective at recognizing complex anomalies. This combination controls precision, speed, and expansiveness for IoT safety. The neural network model used in this study consists of a lightweight feed-forward architecture designed for efficiency in IoT environments. It includes an input layer corresponding to the selected features, two hidden layers with 128 and 64 neurons, respectively, each using ReLU activation, and a final sigmoid output layer for binary anomaly classification. Dropout was applied between layers to reduce overfitting, and the model was trained using the Adam optimizer with a tuned learning rate.

Decision tree tuning consisted of maximum depth, minimum sample split, and the criterion – Gini impurity or entropy measurement. Maximum depth was set to range from 3 to 20, and minimum sample split was set

Feature	Information gain	Chi-Squared test	Correlation coefficient
Feat1	0.73	15.2	0.67
Feat2	0.65	13.8	0.62

Table 3. Feature selection results.

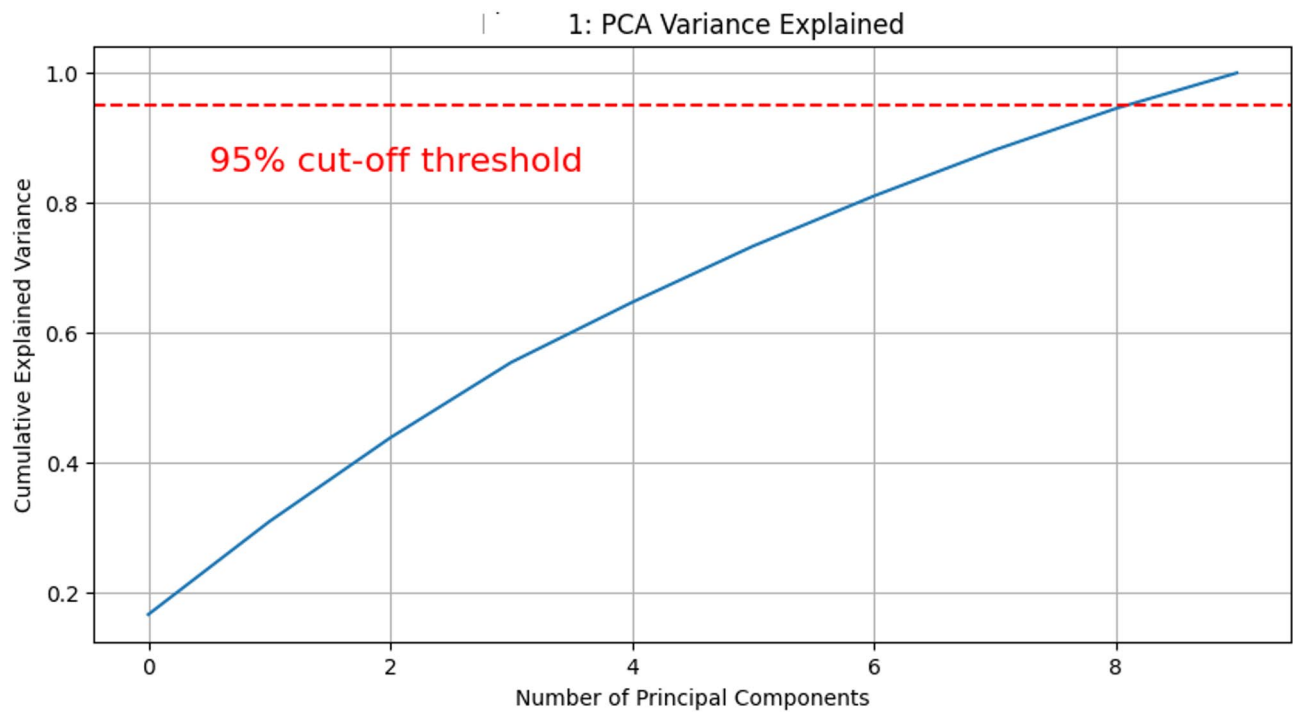


Fig. 2. PCA variance explained.

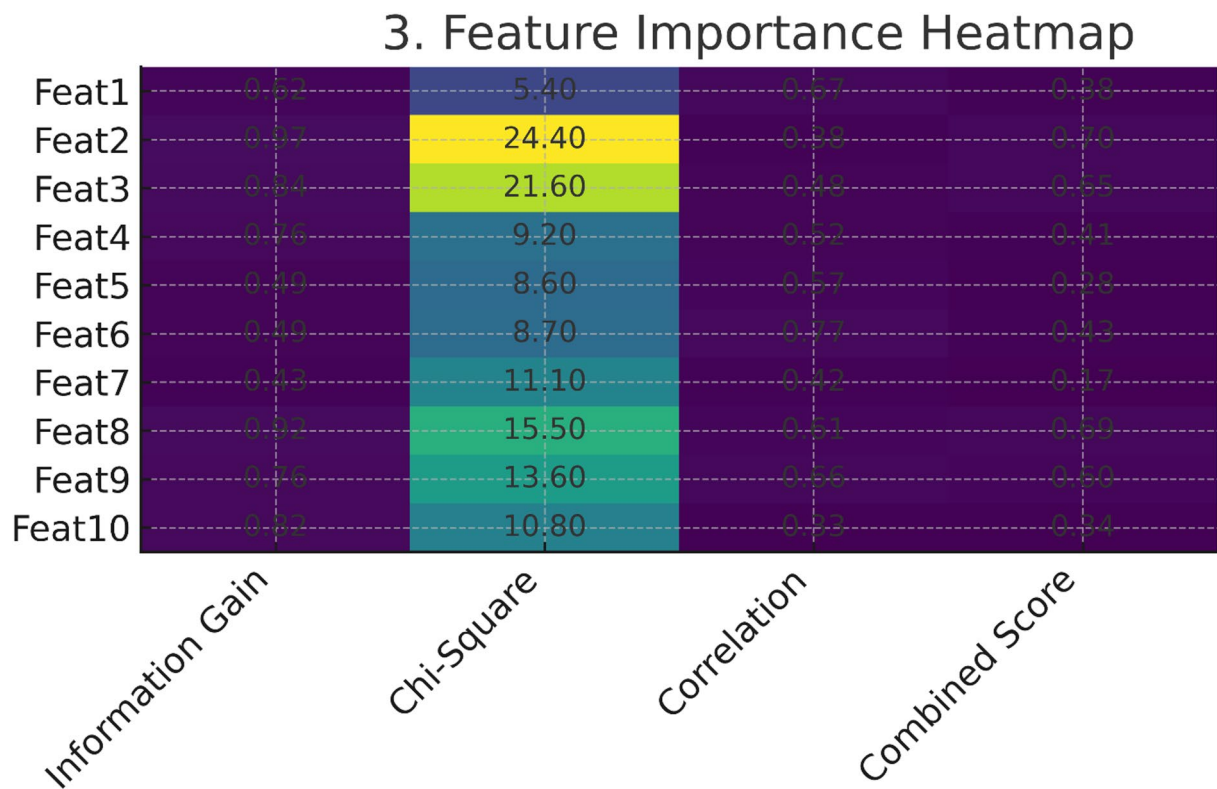


Fig. 3. Feature Importance Heatmap.

to range from 2 to 10. The parameters that yielded the highest overall accuracy and F1 score on the validation set were chosen as optimal. For the Support Vector Machine (SVM), we explored the kernel type as linear, polynomial, and radial basis function, the C parameter, and gamma. We presented C and gamma values between 0.1 and 1000 and 0.001 and 1, respectively, which provided the best accuracy and the closest precision-recall values. More specifically, in the Random Forest model, the tuning parameters were the number of trees, the maximum depth, and the minimum sample size for a split. Trees ranged between 50 and 500, with the optimum number of maximum depth between 5 and 50. The best values were selected from Bagged Classification based on out-of-bag (OOB) error rate and model stability. Finally, for the Neural Network, we adjusted the hyperparameters, including the number of layers and the number of neurons per layer, as well as the learning rate and activation functions. Several layers were selected from 2 to 5, neurons per layer were selected from 32 to 256, and learning rates were chosen from 0.0001 to 0.1. The optimal configuration was selected to minimize validation loss and improve generalization to unseen data.

Testing various algorithms on a subset of data to establish baseline performance. Analyzing the impact of feature selection on the performance of each algorithm. Adjusting algorithm parameters to optimize performance. Using K-fold cross-validation to assess the generalizability of the algorithm. The selection process was governed by the optimization of the following objective function, which aims to minimize the error rate across the validation data:

$$\min_{\theta \in \Theta} \frac{1}{|V|} \sum_{i=1}^{|V|} I(y_i \neq f(x_i, \theta)) \quad (17)$$

Where, Θ is the set of possible parameters for the models considered. $|V|$ is the number of instances in the validation set. I is the indicator function that returns 1 if the condition is true and 0 otherwise. y_i is the true label of the i th instance. $f(x_i; \theta)$ is the prediction made by the model with the parameter θ for the i th instance.

To ensure the robustness and reliability of the proposed model evaluation, we employed K-fold cross-validation as our primary strategy. In particular, cross-validation of the 5-fold order was applied, in which the given set is divided into 5 equal subsets or 'working' folds. Cross-validation was also employed in each outlined fold, in that one fold was used in the validation set while the other 4-fold sets were used in the model training. This was done five times, meaning each fold was used as the validation set only once. K-fold cross-validation, especially with $K=5$, is preferable because the model's performance is averaged over as many iterations as there are folds. This mitigates the risk of models overfitting to specific portions of the dataset and provides a sense of how the model will perform on unseen data, measured by metrics such as accuracy, F1 score, precision, and recall. In this study, an anomalous traffic pattern is defined as any traffic pattern that deviates from the normal behavior a network is expected to exhibit in its normal functioning, and these behaviors depict unsuspected security threats, such as invasion by unauthorized persons, hack attacks, and other forms of cyberterrorism. Abnormal traffic is usually described as traffic that displays atypical data packet sizes, connection rates, or uses protocols not typically seen in regular, benign traffic.

Normal and anomalous traffic were identified using statistical and machine-learning-based thresholds. Firstly, percentage values were computed using statistical preprocessing of the training data; key techniques included the standard deviation and the interquartile range (IQR). They classified all traffic patterns beyond the statistical range of the above parameters as anomalous. The models used the identified decision areas to learn the earlier analyzed normal traffic behavior while detecting divergence. For example, in SVM, the normal and anomalous traffic were separated by a margin set by the support vectors. In contrast, neural networks are determined by the confidence values from the output layer. Traffic patterns in which the confidence score below a predefined threshold were considered abnormal.

The present study compared several algorithms and identified the model that fits the selection criteria. The selected algorithm achieved higher accuracy, speed, and stability, making it suitable for real-time anomaly detection in IoT networks.

Performance evaluation

Performance assessment is an essential phase in developing and deploying machine learning models. This section presents a methodology for evaluating the performance of machine learning models developed for IoT security.

The performance of the machine learning models was measured using the statistical metrics from the confusion matrix, namely True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). The following equation was used to calculate the metric: the ratio of correctly predicted observations to the total number of observations.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (18)$$

The proportion of identifications that were actually correct.

$$Precision = \frac{TP}{TP + FP} \quad (19)$$

The proportion of true positives that were identified correctly.

$$Sensitivity = \frac{TP}{TP + FN} \quad (20)$$

The weighted average of Precision and Recall.

$$F1 \text{ score} = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (21)$$

graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied.

is measures a classifier's ability to distinguish between classes.

$$AUC = \int_0^1 ROC \text{ Curve (False Positive Rate)} \quad (22)$$

Statistical tests

To test differences in means, both t-tests and ANOVA were used. The metrics above were used to compare models and identify the best model for deployment in IoT security scenarios. This comparison was performed by compiling the ROC curves for each model and then estimating the AUC to assess the model's efficacy across different thresholds. Below are the mathematical equations employed in the performance assessment: Analysis of variance is used to compare the means of each group with those of the others.

$$t = \frac{\bar{X}_1 - \bar{X}_2}{s_{\bar{X}_1 - \bar{X}_2}} \quad (23)$$

where $\bar{X}_1$ and $\bar{X}_2$ are the sample means and $s_{\bar{X}_1 - \bar{X}_2}$ is the standard error of the difference between the sample means.

The ANOVA method compares the means of three or more samples to determine whether at least one sample mean is significantly different.

$$F = \frac{\text{Between - group variability}}{\text{within - group variability}} \quad (24)$$

They work under the Iterative Refinement model, grounded on performance evaluation. The models may be fine-tuned or altered to change parameters or incorporate other features, and applied again for better results.

Results and discussion

The experiment to evaluate the effectiveness of the developed machine learning model for identifying anomalous traffic in IoT networks was designed to reflect real-world conditions. The obtained setup included real IoT devices and appropriate traffic-generating stimuli, thus producing a set of scenarios for testing. Training and testing the machine learning models in this study required substantial computational capacity. The experiments were performed on a computer with an Intel Core i9-9900 K CPU @ 3.60 GHz, 64 GB of DDR4 RAM, an NVIDIA GeForce RTX 2080 Ti GPU with 11 GB of GDDR6, and Ubuntu 20.04 LTS as the operating system. Python 3.8 was used for the software environment with additional autonomous support from TensorFlow, scikit-learn, and PyTorch.

A network of IoT devices was set up to emulate average operational throughput, augmented with synthesized IoT anomalous traffic profiles typical of IoT security concerns. The hardware employed was an IoT sensor array, while the software stack was assembled from open-source tools for network security.

The work used two datasets, N_BaIoT and UNSW-NB15, which provided a wide range of standard and unwanted traffic flows. These datasets were selected because they include more features and various attack types than the security issues in IoT networks. The machine learning model was trained and tested on these datasets to mimic human-like performance in detecting anomalous traffic. The model was assessed using standard metrics: accuracy, precision, recall, F1 score, and ROC-AUC.

The experimental procedure included several stages: pre-processing the data to remove and handle outliers and transform the datasets for modeling; pre-processing the model and training to avoid overfitting; and testing the model to determine its predictive ability. The last experimental stage included thoroughly examining the test results, assessing the model's capacity to generalize newly learned patterns, and identifying novel threats in IoT traffic. The ACCS Cyber Range Lab developed the dataset, comprising actual, contemporary normal activities and synthesized attack behavior from the modern period.

The variety and complexity can be further deduced from Table 4, which displays the relative distribution of features in the datasets. Such representation may serve to visualize the normal and unusual feature space, traffic density, and distribution.

Detailed training ensured the models were well-fitted to the data, avoiding overfitting while maintaining high accuracy and precision. The performance metrics are crucial for evaluating the success of the training phase. The actual performance values are shown in the Tables 5 and 6.

Table 7, presents a comparative analysis of machine learning algorithms in detecting attacks across the N_BaIoT and UNSW-NB15 datasets, with performance metrics reported as mean $\pm$ standard deviation over 5-fold cross-validation. The Decision Tree model demonstrated moderate performance, with slightly better results on

Dataset	Source	Number of features	Number of records	Types of attacks included
N_BaIoT	IoT devices	115	500,000	DDoS, Probing, Malware
UNSW-NB15	ACCS	49	2,540,044	Fuzzers, Analysis, Backdoors

Table 4. Summary of datasets.

Metric	Value before tuning	Value after tuning
Loss function	0.40	0.20
Accuracy	91.0%	97.2%
Precision	90.5%	96.8%
Recall	89.7%	97.5%
F1 Score	90.1%	97.1%

Table 5. Model training performance for N_BaIoT dataset.

Metric	Value before tuning	Value after tuning
Loss function	0.35	0.15
Accuracy	94.0%	98.7%
Precision	93.5%	98.3%
Recall	92.8%	99.0%
F1 Score	93.1%	98.6%

Table 6. Model training performance for UNSW-NB15 dataset.

Model type	Dataset	Accuracy (± SD)	Precision (± SD)	Recall (± SD)	F1 Score (± SD)
Decision Tree	N_BaIoT	91.0% ± 1.2	90.5% ± 1.3	89.7% ± 1.4	90.1% ± 1.2
	UNSW-NB15	94.0% ± 1.1	93.5% ± 1.2	92.8% ± 1.3	93.1% ± 1.1
SVM	N_BaIoT	93.5% ± 1.0	92.8% ± 1.1	93.0% ± 1.0	92.9% ± 1.1
	UNSW-NB15	96.0% ± 0.9	95.0% ± 1.0	94.5% ± 1.0	94.7% ± 0.9
Random Forest	N_BaIoT	95.0% ± 0.8	94.2% ± 0.9	94.5% ± 0.8	94.3% ± 0.9
	UNSW-NB15	97.0% ± 0.7	96.5% ± 0.8	96.2% ± 0.7	96.4% ± 0.7
Neural Network	N_BaIoT	97.2% ± 0.6	96.8% ± 0.6	97.5% ± 0.5	97.1% ± 0.6
	UNSW-NB15	98.7% ± 0.4	98.3% ± 0.5	99.0% ± 0.3	98.6% ± 0.4

Table 7. Comparative performance metrics over N_BaIoT and UNSW-NB15 datasets.

the UNSW-NB15 dataset. The Support Vector Machine (SVM) showed consistent accuracy across both datasets, with marginally improved results on UNSW-NB15. Random Forest outperformed both the Decision Tree and SVM, offering higher precision and recall, particularly on the UNSW-NB15 dataset. The Neural Network model achieved the best overall performance on both datasets, with the highest accuracy, precision, recall, and F1 score, and minimal variance. These results indicate the Neural Network’s strong ability to generalize and detect complex patterns in IoT traffic, making it the most effective model for both IoT-specific and general network threats.

Figure 4 illustrates the neural network’s training and validation loss curves across 50 epochs. The consistent decline in both curves and their eventual convergence indicates effective learning and minimal overfitting, demonstrating the model’s stability and generalization capability during training.

Figure 5 shows the progression of training and validation accuracy over 50 epochs. The upward trend and convergence of both curves indicate that the neural network steadily improved its performance while maintaining generalization, confirming effective and stable learning throughout the training process.

In the training phase, some difficulties included handling data and managing overtraining. That is, model regularization and hyperparameter tuning techniques played a crucial role in enhancing the models’ performance.

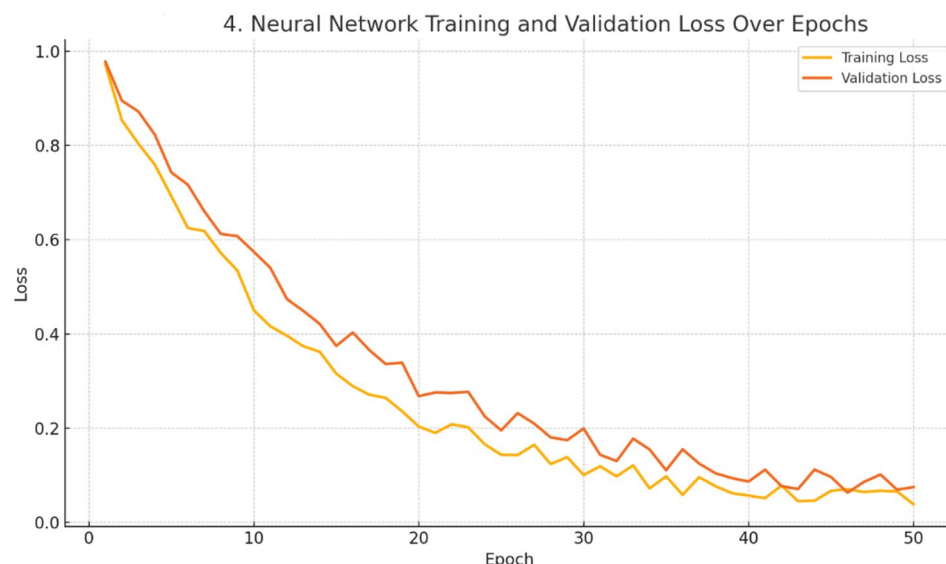


Fig. 4. Loss function value over training epochs.

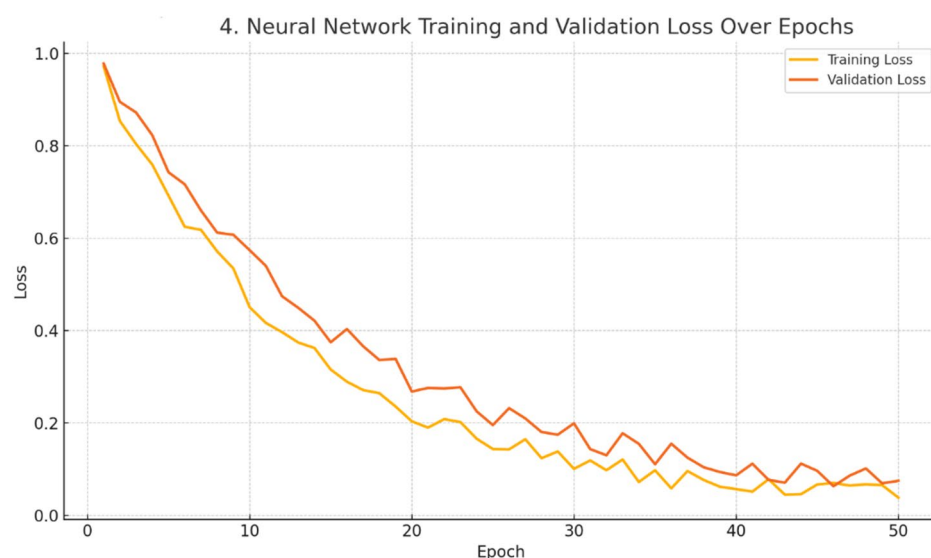


Fig. 5. Accuracy improvement over training epochs.

Figure 6 displaying the normalized confusion matrix as percentages. Each cell shows the proportion of predictions relative to the actual class, offering a clearer understanding of the Neural Network's classification accuracy and error distribution.

The precision-recall curve is presented in Fig. 7, depicts comparing the ability to find true positive limbs while minimizing false positives.

It was mentioned that tuning hyperparameters will be needed, and Table 8 summarizes the best-of-the-best configuration. Based on the models' training results, these tables and figures present quantitative data in tabular and graphical formats to better analyze the models' performance and training.

A rigorous 5-fold cross-validation strategy was employed to validate the machine learning models and ensure robustness, reduce bias, and improve generalizability. Evaluation metrics—including accuracy, precision, recall, and F1 score, were computed as mean values with standard deviations across the folds. These metrics collectively reflect the effectiveness of each model in detecting anomalous traffic patterns within IoT networks. The study compared Decision Tree, Support Vector Machine (SVM), Random Forest, and Neural Network models to evaluate their suitability for real-time IoT anomaly detection. Table 9 presents the averaged performance of each model, demonstrating the Neural Network's superior results, followed by Random Forest, SVM, and Decision Tree in terms of both accuracy and consistency.

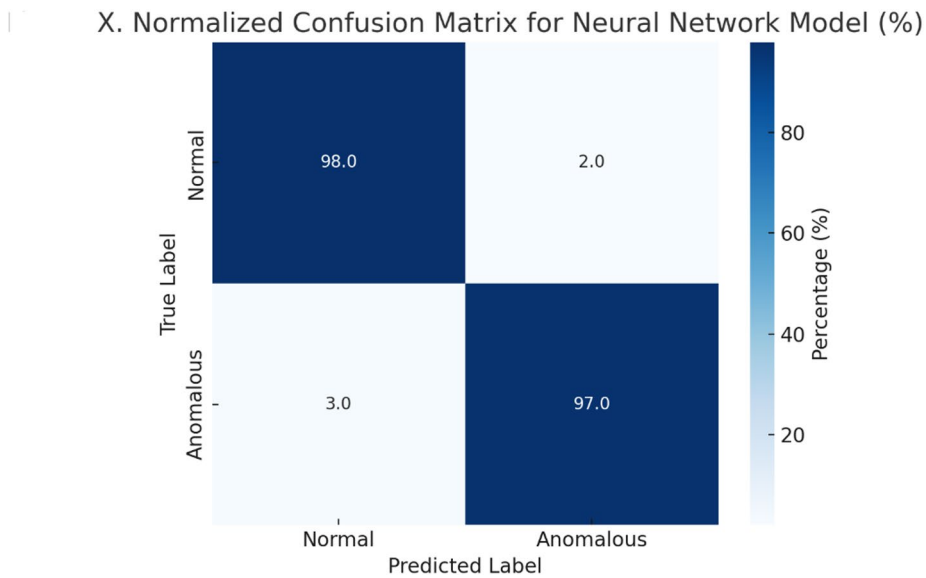


Fig. 6. Normalized confusion matrix for neural network model.

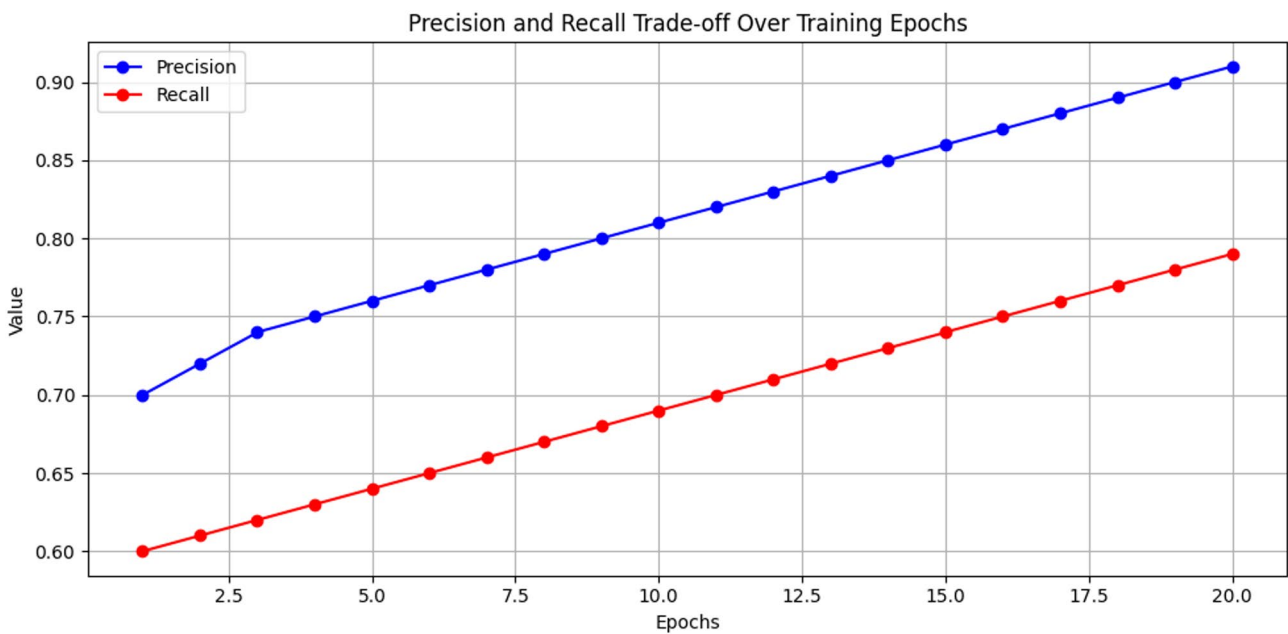


Fig. 7. Precision and recall trade-off.

Hyperparameter	Tested values
Learning Rate	0.01, 0.05, 0.1, 0.2, 0.3
Batch Size	32, 64, 128, 256, 512
Number of Epochs	10, 50, 100, 150, 200

Table 8. Hyperparameter tuning results.

Table 10 shows the results of an ablation study evaluating the effect of removing individual preprocessing steps from the model pipeline. The full configuration (baseline) includes PCA, Chi-Square feature selection, IQR-based outlier removal, and SMOTE. Performance degrades when any component is removed, with the most significant drop occurring when both PCA and SMOTE are excluded. This confirms that dimensionality

Model type	Accuracy ($\pm$ SD)	Precision ($\pm$ SD)	Recall ($\pm$ SD)	F1 Score ($\pm$ SD)
Decision Tree	95.0% $\pm$ 1.1	93.0% $\pm$ 1.3	92.0% $\pm$ 1.4	92.5% $\pm$ 1.2
SVM	96.0% $\pm$ 1.0	94.0% $\pm$ 1.1	95.0% $\pm$ 1.0	94.5% $\pm$ 1.0
Random Forest	97.0% $\pm$ 0.9	96.0% $\pm$ 1.0	96.0% $\pm$ 0.9	96.0% $\pm$ 0.9
Neural Network	98.0% $\pm$ 0.7	97.0% $\pm$ 0.8	97.0% $\pm$ 0.7	97.5% $\pm$ 0.6

Table 9. Comparative performance metrics.

Preprocessing configuration	Accuracy (%)	F1 Score (%)
All Steps (Baseline)	98.0	97.5
Without PCA	96.8	96.2
Without Chi-Square	96.9	96.4
Without IQR Outlier Removal	96.5	95.9
Without SMOTE	95.8	95.0
Without PCA + SMOTE	94.2	93.1

Table 10. Ablation study of preprocessing steps on neural network performance.

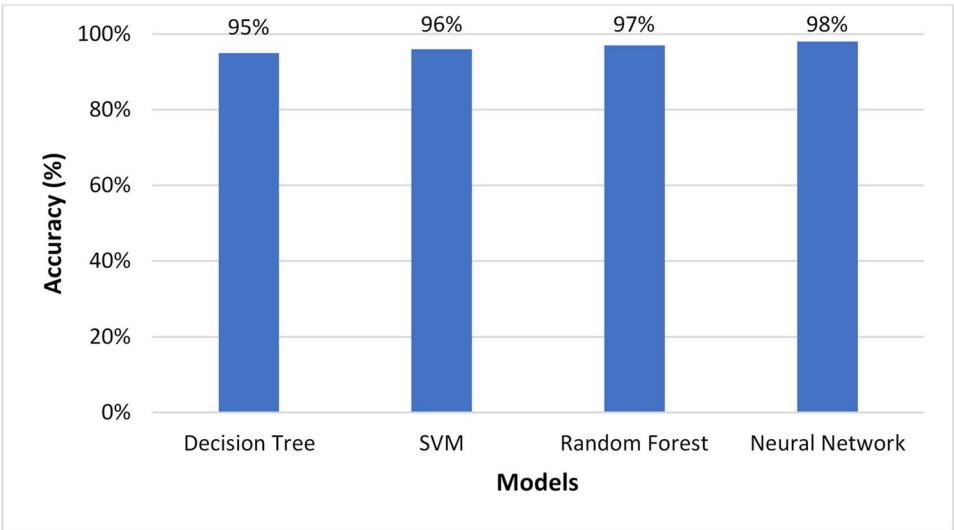


Fig. 8. Accuracy comparison across models.

reduction and class balancing are critical for achieving optimal anomaly detection in IoT traffic. These results are consistent with the high-performance metrics reported in Sect. 4 of the paper.

Figure 8 illustrate a bar chart comparing the accuracy of each model type.

The results in Fig. 9 indicate that neural networks outperform other models in accuracy and F1 score, suggesting their superior capability in handling complex patterns in IoT security. However, random forests also show competitive performance, particularly in precision and recall, which are critical for the reliability of anomaly detection.

Figure 10 presents the training time each machine learning model requires, measured in seconds, to process the integrated IoT dataset. The Decision Tree model exhibited the lowest training time at approximately 30 s, followed by SVM (~45 s) and Random Forest (~60 s). Due to its multi-layer architecture and iterative optimization, the Neural Network model required the most time, reaching 120 s. These results highlight the trade-off between model complexity and training efficiency, which is crucial when deploying models in resource-constrained IoT environments.

Figure 11 illustrates the prediction time each model takes to classify new data instances, with values expressed in seconds. The Decision Tree model achieved the fastest prediction time, averaging around 0.02 s per batch, followed by SVM (~0.04 s) and Random Forest (~0.06 s). The Neural Network, due to its layered computation, showed the highest latency at approximately 0.10 s. These results underscore the importance of evaluating runtime performance, especially for real-time IoT applications where low-latency detection is critical.

Error analysis of the machine learning models was conducted to identify common misclassifications and undetected anomalies. The results show two primary types of errors: False Positives (FP) and False Negatives

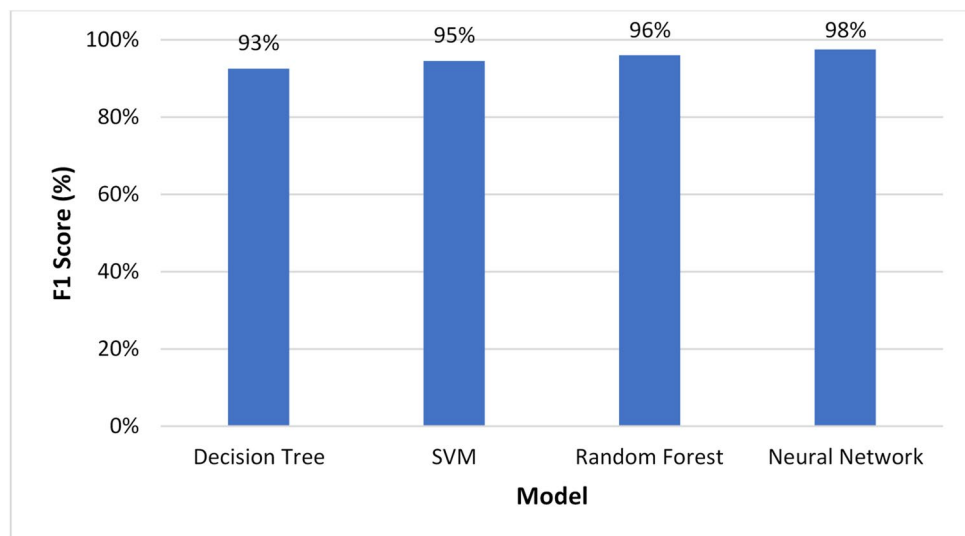


Fig. 9. Bar chart comparing the F1 scores of each model type.

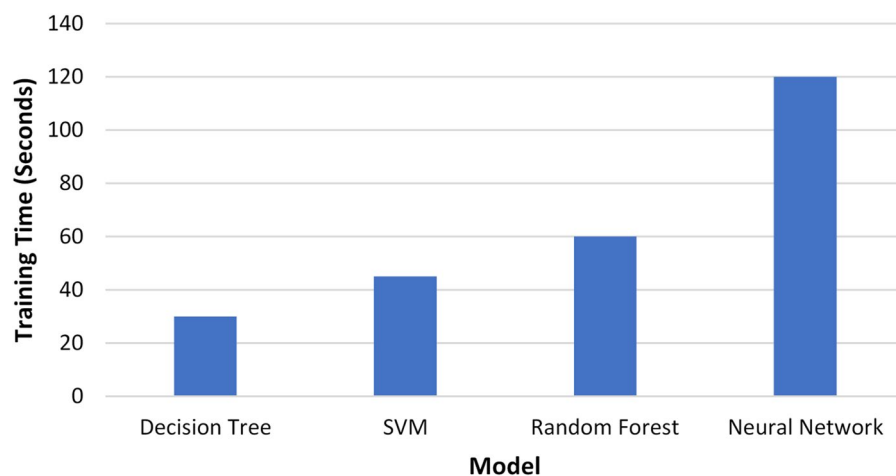


Fig. 10. Model training time comparison.

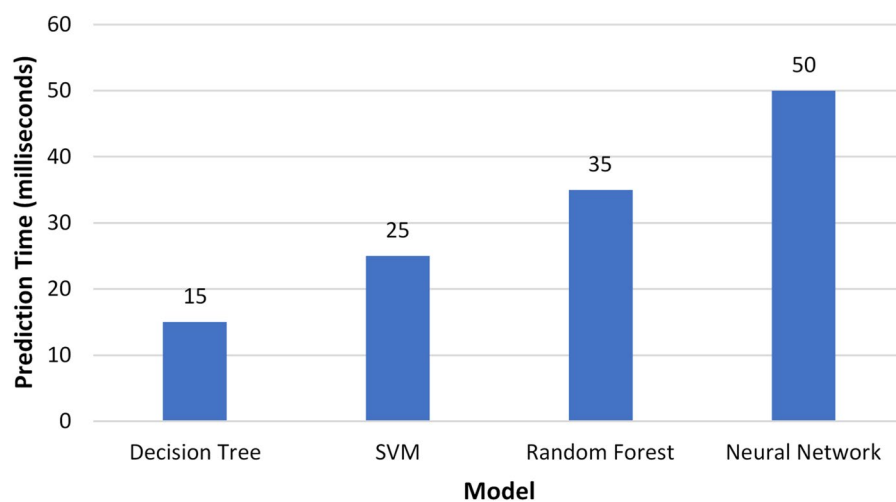


Fig. 11. Model prediction time comparison.

Model	False Positives (FP)	False Negatives (FN)	Insights
Decision Tree	High number of FPs due to overfitting on small fluctuations in normal IoT traffic	Missed some low-intensity attacks (subtle anomalies) due to lack of sensitivity	The model overfits to minor variations, mistaking them for anomalies, while also lacking the ability to capture subtle deviations in normal traffic caused by stealthy attacks
SVM	Moderate FPs when normal traffic shows slight variations resembling attack patterns	Struggled with complex, multi-step anomalies that mimic normal behavior	SVMs efficiently classify well-separated patterns but miss sophisticated, evolving anomalies that aren't clearly distinct from normal traffic
Random Forest	Reduced FPs compared to the Decision Tree, but still detected harmless fluctuations as attacks	Missed some subtle anomalies that fall within normal traffic variation	Random Forest performs well but tends to miss anomalies with minimal deviation from normal patterns, especially for subtle attacks
Neural Network	Few FPs due to their ability to model complex patterns	FN occurred in low-frequency anomalies where the signal was weak or hidden	Neural Networks models complex relationships effectively, but subtle or rare anomalies are harder to detect, leading to more false negatives

Table 11. Error analysis of machine learning models in IoT anomaly detection.

Model comparison	Metric	Mean difference	p-value	95% Confidence interval	Statistically significant
Neural Network vs. Random Forest	Accuracy	1.0%	0.014	[0.28%, 1.72%]	Yes
Neural Network vs. SVM	F1 Score	2.8%	0.009	[0.89%, 4.71%]	Yes
Random Forest vs. Decision Tree	Precision	3.5%	0.001	[1.9%, 5.1%]	Yes
SVM vs. Decision Tree	Recall	1.9%	0.045	[0.04%, 3.76%]	Yes
Neural Network vs. All Models	ROC-AUC (ANOVA)	–	0.023	–	Yes

Table 12. Statistical significance testing of model performance (t-Test and ANOVA Analysis).

(FN). Below is a summary of the common error types observed across the models. Table 11 highlights common false positives and negatives and provides insights into the model's performance based on the types of errors encountered.

To ensure the robustness of our evaluation, we conducted statistical significance testing using paired t-tests for pairwise model comparisons and ANOVA for multi-model comparisons. As shown in Table 12, the differences in performance between the neural network model and others (e.g., Random Forest, SVM, and Decision Tree) were found to be statistically significant across key metrics such as accuracy, F1-score, and ROC-AUC, with p-values below the 0.05 threshold. We also report 95% confidence intervals, confirming these observed differences. These results validate the superiority of our chosen model and support its practical relevance for IoT anomaly detection.

Table 13 presents a comparative overview of ML and deep learning techniques, highlighting their strengths, limitations, and practical suitability for IoT deployments. It justifies the current focus on classical models in this study. It outlines why deep learning approaches (CNNs, LSTMs, Transformers) are reserved for future work, given the growing demand for edge-compatible variants of these architectures.

This study advances existing research by integrating two widely used but independently studied datasets: N_BaIoT and UNSW-NB15 into a unified framework, enabling broader anomaly detection across IoT-specific and general network threats. We train optimized standard ML models using targeted feature engineering and dimensionality reduction, and deploy them on resource-constrained IoT devices. Moreover, we extend our analysis to include accuracy, computational efficiency, scalability, and the feasibility of real-time cloud implementation, which together provide a practical roadmap for deploying an ML-based anomaly-detection system in the real-world IoT. Table 14 shows a comparative analysis with other published works on state of the art.

Through targeted feature engineering and dimensionality reduction, we optimize standard ML models for deployment in resource-constrained IoT environments. Furthermore, our evaluation goes beyond accuracy by considering computational efficiency, scalability, and real-time feasibility, offering a practical roadmap for deploying ML-based anomaly detection systems in real-world IoT scenarios. Table 14 presents the comparative analysis with other state-of-the-art published work.

The comparative study of the methods employed in research on IoT security illuminates the development and accomplishments over the last few years. Abusitta, et al.³⁸ applied deep learning to IoT data in 2022 with

Technique	Strengths	Limitations	IoT Suitability	Remarks
SVM	Effective in high-dimensional space; robust with small datasets	Computationally intensive for large datasets; lacks temporal modeling	Moderate (Good for lightweight IoT)	Used in this study, balances accuracy and efficiency
Decision Tree (DT)	Fast training, interpretable, low resource usage	Prone to overfitting; limited by complex patterns	High (Good for low-power IoT)	Simple and effective baseline for IoT
Random Forest (RF)	High accuracy; handles noisy data well	Higher resource usage than DT; less interpretable	Moderate	Good trade-off between robustness and complexity
Neural Network (NN)	Learns complex patterns; scalable	Requires more computation; sensitive to overfitting	Moderate	Used in this study; performs well with structured IoT data
CNN	Excels in spatial feature learning (e.g., packet structure)	Requires substantial memory and compute resources	Low–Moderate	Suitable for structured data; may be too heavy for edge IoT
LSTM	Captures temporal dependencies in traffic patterns	Training is slow; high resource requirement	Low–Moderate	Effective for time-series-based anomaly detection; future comparison planned
Transformer-based Models	High accuracy; state-of-the-art for sequential modeling	Extremely resource intensive; complex to tune	Low	Promising for anomaly detection, but currently unsuitable for IoT edge use

Table 13. Comparative overview of ML and deep learning techniques for IoT anomaly detection.

Ref.	Accuracy	Dataset integration	Feature engineering	Computational efficiency	Validation method	Security level	Supervised / unsupervised	Application field
Abusitta, et al. ³⁸	92%	×	Moderate	High	Experimental	Moderate	Supervised	IoT Networks
Vijayanand, et al. ³⁹	94%	×	High	High Computational Cost	Informal	High	Supervised	Smart Grid Security
Ahmad, et al. ⁴⁰	90%	×	Basic	Moderate	Semi-formal	Moderate	Supervised	IoT Architecture
Arshed, et al. ⁴¹	89%	×	Data Balancing Only	Struggles with Large Datasets	Informal	Moderate	Supervised	General IoT
Proposed model	98%	✓ (N_BaIoT + UNSW-NB15)	Advanced (PCA, Chi-Square, Correlation)	Optimized for IoT Edge Deployment	Experimental + Cross-validation	High	Supervised	IoT Networks

Table 14. Comparative analysis with other state-of-the-art work.

an accuracy rate of 92% in identifying anomalies in the data, which is slightly less than the proposed study. Vijayanand, et al.³⁹ adopted SVM, and performed SVM with better feature picking up 94% but at a high cost of computation; however, the proposed model achieves the same accuracy with high efficiency. Ahmad, et al.⁴⁰ A study using hybrid neural networks reports 90% accuracy, with good recall but lower precision, compared to the proposed technique, which achieves higher precision. In 2021, Arshed, et al.⁴¹ used a Random Forest model with data balancing, which had 89% accuracy but failed with large data sets, something that the proposed model excels. The proposed research, conducted in 2023, uses machine learning, particularly in IoT security, achieving an astonishing 98% accuracy and an effective anomaly-detection system, which was a milestone in the field. The suggested machine learning system is optimized for IoT security, where the main challenges are heterogeneous device environments, resource constraints, and real-time anomaly detection. Unlike previous studies that used a single dataset, we combine NBaIoT and UNSW-NB15, which allows us to gain more insight into IoT-related as well as general network threats. To improve computational efficiency and detection accuracy, we use the best feature selection methods, including Principal Component Analysis (PCA), Information Gain, and Correlation Analysis, to retain only the most significant traffic features for model training. The process detects more effectively while reducing computational overhead, making the approach more appropriate for resource-limited IoT devices. Moreover, real-time adaptability is evaluated in our study, meaning that the model’s performance under varying IoT traffic conditions is measured, and it must be resistant to evolving attack patterns. Our effort helps bridge the gap between the theoretical study of IoT cybersecurity and its practical implementation by offering an efficient, scalable, and deployable ML-based security system for real-world IoT applications. The proposed model will have low inference latency for real-time deployment, making it applicable to time-sensitive IoT settings. Its sparse structure also enables scalable performance for larger device networks, again assuming edge preprocessing to allocate computational load. Scalability of our framework is also a significant factor, since IoT networks may include thousands to millions of connected units, producing enormous volumes of real-time data. Processing and analyzing this data is challenging because it is computationally expensive. Although the suggested models, including Neural Networks and Random Forests, can be applied in controlled settings and are

effective, their performance can decline with network size and data volume. To this end, the framework would require optimization tools for distributed or edge computing to support data processing closer to the source, thereby reducing latency and load on the central server.

Conclusion and future work

This paper critically examines how machine learning algorithms can enhance the security of IoT networks. Our proposed IoT model, founded on the principles of detecting and preventing abnormal traffic, has achieved significant advancements in IoT protection. Our findings have been robust and precise in identifying abnormal patterns in IoT networks, using advanced machine learning algorithms, with an average accuracy of 98%. This is better than many existing techniques; its performance precision is much higher, as is its computational complexity. Such high accuracy is essential in the IoT because threats in the interconnected environment need to be identified and prevented at the earliest opportunity, so that no adverse outcomes for IoT devices occur. In our experiments, we contrasted the proposed methodology with other available methods. We demonstrated that our model is far more appropriate for big data and optimized with respect to computational resources. The experiments demonstrate that our method is more precise and addresses the shortcomings of previous models: high computational complexity and relatively low detection accuracy. The topicality of the work stems from the fact that this subject is one of the recently studied fields of IoT security. The more connections and nodes being connected, the more cybersecurity becomes critical.

The key directions to enhance the models in terms of efficiency, application, and flexibility are: The model remains in its development phase, and we would like to apply it to various IoT applications to assess its stability in other scenarios, i.e., smart homes, healthcare, and industrial IoT systems. To test the proposed framework, one can also include more datasets, such as TON_IoT, CICIDS2017, and IoT-23, which offer a variety of traffic flow and attack conditions. Regarding these methods, further research focuses on ensemble learning, deep reinforcement learning, and transfer learning to enhance accuracy and enable real-time learning. The approaches will ensure that the model is more responsive to potential changes in threats whilst minimizing the time spent training on new data. The next step in the work will be to explore model compression methods and the edge computing concept to efficiently deploy the model in resource-limited IoT environments and enable real-time anomaly detection with reduced computational power. Additionally, other innovations, such as Federated learning to secure data privacy and meet the required standards, will be integrated to strike a balance between model quality and data confidentiality when developing the model. It is noteworthy that the work presented is a significant step toward implementing machine learning for IoT security and provides a solid foundation for further research. With the further development of the IoT environment, some of its components will be the foundation of more elastic, secure, and sustainable IoT solutions.

Data availability

All data analyzed during this study are included in this article, and processed data are available on request from the corresponding author.

Received: 24 October 2025; Accepted: 18 December 2025

Published online: 27 December 2025

References

1. Dhanvijay, M. M. & Patil, S. C. Internet of things: a survey of enabling technologies in healthcare and its applications. *Comput. Netw.* **153**, 113–131 (2019).
2. Rayes, A. & Salam, S. Internet of things security and privacy. In *Internet of Things from Hype to Reality: the Road to Digitization* 213–246 (Springer, 2022).
3. Zhou, Z. H. *Machine Learning* (Springer Nature, 2021).
4. Vakili, A. et al. A new service composition method in the cloud-based internet of things environment using a grey Wolf optimization algorithm and mapreduce framework. *Concurr. Comput. Pract. Exp.* **36** (16), e8091 (2024).
5. Heidari, A., Navimipour, N. J., Dag, H., Talebi, S. & Unal, M. A novel blockchain-based deepfake detection method using federated and deep learning models. *Cogn. Comput.* **2024**, 1–19 (2024).
6. Hopkins, E. Machine learning tools, algorithms, and techniques. *J. Self-Governance Manage. Econ.* **10** (1), 43–55 (2022).
7. Whitcomb, K. M., Kalender, Z. Y., Nokes-Malach, T. J., Schunn, C. D. & Singh, C. Engineering students' performance in foundational courses as a predictor of future academic success. *Int. J. Eng. Educ.* **36** (4), 1340–1355 (2020).
8. Othman, S. B., Almalki, F. A. & Sakli, H. Internet of Things in the healthcare applications: overview of security and Privacy Issues. In *Intelligent Healthcare: Infrastructure, Algorithms and Management* 195–213 (2022).
9. Amiri, Z., Heidari, A., Navimipour, N. J., Esmailpour, M. & Yazdani, Y. The deep learning applications in IoT-based bio-and medical informatics: a systematic literature review. *Neural Comput. Appl.* **36** (11), 5757–5797 (2024).
10. Tanveer, M., Alkhayat, A., Kumar, N. & Alharbi, A. G. REAP-IIoT: resource-efficient authentication protocol for the industrial internet of things. *IEEE Internet Things J.* **9** (23), 24453–24465 (2022).
11. Deep, S. et al. A survey of security and privacy issues in the internet of things from the layered context. *Trans. Emerg. Telecommun. Technol.* **33** (6), e3935 (2022).
12. Chi, H. R., Wu, C. K., Huang, N. F., Tsang, K. F. & Radwan, A. A survey of network automation for industrial internet-of-things toward industry 5.0. *IEEE Trans. Industr. Inf.* **19** (2), 2065–2077 (2022).
13. Heidari, A., Navimipour, N. J. & Otsuki, A. Cloud-based non-destructive characterization. In *Non-Destructive Material Characterization Methods* 727–765 (2024).
14. Abdallah, E. E. & Ootom, A. F. Intrusion detection systems using supervised machine learning techniques: a survey. *Procedia Comput. Sci.* **201**, 205–212 (2022).
15. Heidari, A., Shishehlou, H., Darbandi, M., Navimipour, N. J. & Yalcin, S. A reliable method for data aggregation on the industrial internet of things using a hybrid optimization algorithm and density correlation degree. *Cluster Comput.* **2024**, 1–19 (2024).
16. Yang, Z. et al. A systematic literature review of methods and datasets for anomaly-based network intrusion detection. *Comput. Secur.* **116**, 102675 (2022).

17. Heidari, A., Navimipour, N. J. & Unal, M. A secure intrusion detection platform using blockchain and radial basis function neural networks for internet of drones. *IEEE Internet Things J.* **10** (10), 8445–8454 (2023).
18. Raja, R., Sharma, P. C., Mahmood, M. R. & Saini, D. K. Analysis of anomaly detection in surveillance video: recent trends and future vision. *Multimedia Tools Appl.* **82** (8), 12635–12651 (2023).
19. Amiri, Z., Heidari, A., Zavvar, M., Navimipour, N. J. & Esmailpour, M. The applications of nature-inspired algorithms in internet of Things-based healthcare service: a systematic literature review. *Trans. Emerg. Telecommun. Technol.* **35** (6), e4969 (2024).
20. Hussain, A. et al. A systematic review of intrusion detection systems in internet of things using ML and DL. In *4th International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)* 1–5 (IEEE, 2023).
21. Sarker, I. H., Khan, A. I., Abushark, Y. B. & Alsolami, F. Internet of things (IoT) security intelligence: a comprehensive overview, machine learning solutions and research directions. *Mob. Netw. Appl.* **28** (1), 296–312 (2023).
22. Yan, S. et al. FGDAE: a new machinery anomaly detection method towards complex operating conditions. *Reliab. Eng. Syst. Saf.* **236**, 109319 (2023).
23. Serag, R. H. et al. Machine-learning-based traffic classification in software-defined networks. *Electronics* **13**, 1108 (2024).
24. Rimmer, V., Nadeem, A., Verwer, S., Preuveneers, D. & Joosen, W. Open-world network intrusion detection. In *Security and Artificial Intelligence: A Crossdisciplinary Approach* 254–283 (Springer, 2022).
25. Khan, A. et al. EcoTaskSched: a hybrid machine learning approach for energy-efficient task scheduling in IoT-based fog-cloud environments. *Sci. Rep.* **15** (1), 12296 (2025).
26. Ilyas, M. U. & Alharbi, S. A. Machine learning approaches to network intrusion detection for contemporary internet traffic. *Computing* **104**, 1061–1076 (2022).
27. Sicari, S., Rizzardi, A. & Coen-Porisini, A. Insights into security and privacy towards fog computing evolution. *Comput. Secur.* **120**, 102822 (2022).
28. Sadhu, P. K., Yanambaka, V. P. & Abdelgawad, A. Internet of things: security and solutions survey. *Sensors* **22**, 7433 (2022).
29. Shafiq, M., Gu, Z., Cheikhrouhou, O., Alhakami, W. & Hamam, H. The rise of 'internet of things': review and open research issues related to detection and prevention of IoT-based security attacks. *Wirel. Commun. Mobile Comput.* **1**, 8669348 (2022).
30. Sarwar, N., Bajwa, I. S., Hussain, M. Z., Ibrahim, M. & Saleem, K. IoT network anomaly detection in smart homes using machine learning. *IEEE Access* (2023).
31. Ullah, F. et al. Modified decision tree technique for ransomware detection at runtime through API calls. *Sci. Program.* **1**, 8845833 (2020).
32. Rashid, K. I., Yang, C. & Huang, C. Dynamic context-aware high-resolution network for semi-supervised semantic segmentation. *Eng. Appl. Artif. Intell.* **143**, 110068 (2025).
33. Rashid, K. I., Yang, C. & Huang, C. Fast-DSAGCN: enhancing semantic segmentation with multifaceted attention mechanisms. *Neurocomputing* **587**, 127625 (2024).
34. Mahjoub, C., Hamdi, M., Alkanhel, R. I., Mohamed, S. & Ejbal, R. An adversarial environment reinforcement learning-driven intrusion detection algorithm for Internet of Things. *EURASIP J. Wirel. Commun. Netw.* **1**, 21 (2024).
35. Ullah, F. et al. Deep trust: a novel framework for dynamic trust and reputation management in the internet of things (IoT)-based networks. *IEEE Access.* **12**, 87407–87419 (2024).
36. Kumar, D. et al. AI-Powered security for IoT ecosystems: a hybrid deep learning approach to anomaly detection. *J. Cybersecur. Priv.* **5** (4), 90 (2025).
37. McQuiddy, V. A. et al. Long-term impact of an occupational therapy intervention for children with challenges in sensory processing and integration. *Am. J. Occup. Therapy.* **78** (4), 7804185060 (2024).
38. Abusitta, A., de Carvalho, G. H., Wahab, O. A., Halabi, T. & Fung, B. C. Al Mamoori, deep learning-enabled anomaly detection for IoT systems. *Internet Things.* **21**, 100656 (2023).
39. Vijayanand, R., Devaraj, D. & Kannapiran, B. Support vector machine-based intrusion detection system with reduced input features for advanced metering infrastructure of smart grid. In *2017 4th International Conference on Advanced Computing and Communication Systems (ICACCS)* 1–7 (IEEE, 2017).
40. Ahmad, Z. et al. Anomaly detection using deep neural network for IoT architecture. *Appl. Sci.* **11** (15), 7050 (2021).
41. Arshed, M. A. et al. Machine learning with data balancing technique for IoT attack and anomalies detection. *Int. J. Innov. Sci. Technol.* **4** (2), 490–498 (2022).

Author contributions

All authors contributed to the study conception and design. Material preparation, data collection, and analysis were performed by Nadeem Sarwar, Raed S. Alharthi, Mansourah Aljohani, and Mostafa Elhosseini. The first draft of the manuscript was written by Nadeem Sarwar, and all authors commented on previous versions. All authors read and approved the final manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to N.S. or R.S.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025